

evaluation

April 11, 2026

1 Evaluation: Scenario-Based vs Microbenchmark Benchmarking

Thesis context: Meaningful performance evaluation of serverless applications requires benchmarks that reflect realistic usage patterns rather than isolated micro-level measurements. This notebook evaluates whether deriving workload scenarios from OpenAPI specifications produces richer, more actionable performance insights than traditional microbenchmark-style load testing with wrk2.

Testing Points: - **TP-A:** Per-Operation Performance Characterization (insight depth + saturation detection + data validity) - **TP-B:** Execution Phase Granularity (cold vs steady per-operation penalties) - **TP-C:** Scenario Shape Sensitivity (read-heavy vs mixed vs lifecycle) - **TP-D:** Cross-Framework Decision Quality (Spring vs Quarkus)

Results root: /home/bakhtia/Documents/thesis/harness-evaluation/evaluation/results

1.1 0. Data Loading & Parsing

Auto-discover all benchmark results under results/. Each result lives at:

results/<app>/<treatment>/<scenario>/<phase>/R<rate>/run-<N>/

Discovered 14 result directories

	treatment	app	scenario	phase	runs
0	slsbench	quarkus-jvm	lifecycle	steady	2
1	slsbench	spring	lifecycle	steady	2
2	slsbench	spring	mixed	cold	1
3	slsbench	spring	mixed	steady	1
4	slsbench	spring	read-heavy	steady	1
5	wrk2	quarkus-jvm	lifecycle	steady	2
6	wrk2	spring	lifecycle	steady	2
7	wrk2	spring	mixed	cold	1
8	wrk2	spring	mixed	steady	1
9	wrk2	spring	read-heavy	steady	1

wrk2 aggregate rows: 7

slsbench per-step rows: 92

slsbench aggregate rows: 14

first-response rows: 8

container-stats rows: 7

raw latency records: 39250

```

slsbench step columns: ['app', 'treatment', 'scenario', 'phase', 'rate', 'run',
'path', 'stage', 'operation_id', 'avg_us', 'max_us', 'sum_us', 'count',
'responses_2xx', 'responses_non2xx', 'p50_us', 'p95_us', 'p99_us',
'responses_2xx_raw', 'responses_non2xx_raw']

```

```

      app treatment  scenario  phase  rate  run
path      stage  operation_id \
0 quarkus-jvm slsbench lifecycle steady  200  1 ../results/quarkus-
jvm/slsbench/lifecycle/stea... lifecycle      addOwner
1 quarkus-jvm slsbench lifecycle steady  200  1 ../results/quarkus-
jvm/slsbench/lifecycle/stea... lifecycle  addPetToOwner
2 quarkus-jvm slsbench lifecycle steady  200  1 ../results/quarkus-
jvm/slsbench/lifecycle/stea... lifecycle      addVet

```

```

      avg_us  max_us  sum_us  count  responses_2xx  responses_non2xx  p50_us
p95_us  p99_us  responses_2xx_raw  responses_non2xx_raw
0      609   35548  2897642   4757           4757           0   295.0
2288.00  5900.56           4757.0           0.0
1         0         0         0         0           0           0   NaN
NaN      NaN      NaN      NaN      NaN
2      616   23013  440092    714           714           0   292.0
2063.35  6121.90           714.0           0.0

```

1.2 TP-A: Per-Operation Performance Characterization

Hypothesis: OpenAPI-derived scenario benchmarks reveal per-operation performance variance and per-operation saturation points that aggregate microbenchmarks mask.

This TP merges three claims: 1. Per-operation latency variance is hidden by aggregates (>10x spread) 2. Under increasing load, operations saturate at different rates 3. wrk2 hardcoded IDs produce elevated error rates (data validity)

Table A1: slsbench Per-Operation Latency Breakdown (lifecycle, R=500, steady, n=1 run(s))

Active operations: 11 (excluded 8 with zero traffic)

Max/Min variance ratio: 9.1x

	Avg (us)	Std Dev	p50 (us)	p95 (us)	p99 (us)	N	% of
Flow							
operation_id							
listPets	15036.0	NaN	12048.0	35085.6	65933.1	169	32.
2							
listVisits	11083.0	NaN	9606.0	20244.9	39905.6	78	23.
8							
addPetToOwner	2975.0	NaN	1726.0	9157.6	17057.6	393	6.
4							
listVets	2930.5	2184.3	1777.8	8932.2	19778.6	1304	6.
3							

addVet ↳4	2503.5	1518.2	1396.5	7226.5	19873.0	1927	5.
addVisitToOwner ↳2	2443.0	NaN	1251.5	8457.5	16808.1	196	5.
getOwner ↳2	2416.0	NaN	928.0	6052.8	13101.5	282	5.
getVisit ↳2	1949.0	NaN	1291.0	5735.1	7593.5	118	4.
addOwner ↳0	1860.5	417.9	862.0	6162.7	11923.1	12869	4.
getVet ↳9	1805.0	NaN	891.5	6376.2	14926.4	132	3.
getOwnersPet ↳6	1658.0	NaN	1048.0	5917.5	9361.6	196	3.

wrk2 aggregate avg *flow completion time* at same rate: 11,995,000 us (11,995.0 ms)

Note: wrk2 measures total chain time (all steps combined), NOT individual step latency.

wrk2 reports ONE number; slsbench reveals 11 distinct per-step latencies ranging from 1658 to 15036 us

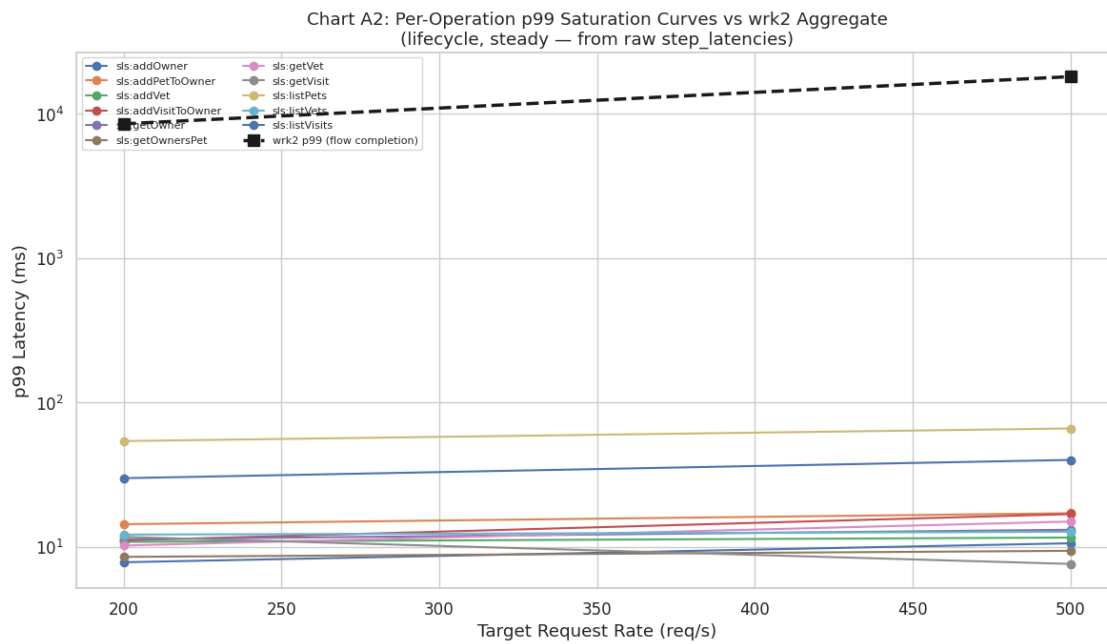


Table A3: Data Validity Comparison (lifecycle, steady)

Treatment	Error Rate (%)		Non-2xx		Total Requests	
	slsbench	wrk2	slsbench	wrk2	slsbench	wrk2

Rate						
200	0.41	9.86	36	457	8882	4635
500	0.21	12.62	37	750	17664	5948

VARIANCE CHECK: max/min latency ratio = 8.6x (threshold: >2.0x)

Fastest: addOwner = 1614.2 us

Slowest: listPets = 13806.5 us

-> CONFIRMED: per-operation variance > 2.0x

SATURATION CHECK (from raw step_latencies):

addVisitToOwner: max consecutive p99 jump = 1.5x

getVet: max consecutive p99 jump = 1.5x

addOwner: max consecutive p99 jump = 1.4x

listVisits: max consecutive p99 jump = 1.3x

listPets: max consecutive p99 jump = 1.2x

addPetToOwner: max consecutive p99 jump = 1.2x

getOwner: max consecutive p99 jump = 1.2x

getOwnersPet: max consecutive p99 jump = 1.1x

addVet: max consecutive p99 jump = 1.1x

listVets: max consecutive p99 jump = 1.0x

getVisit: max consecutive p99 jump = 0.6x

Jump spread (max/min): 2.3x (threshold: >1.5x)

-> CONFIRMED

VALIDITY CHECK: wrk2 error rate = 11.41% (threshold: >0.5%)

slsbench error rate = 0.27%

-> CONFIRMED: wrk2 produces elevated error rates

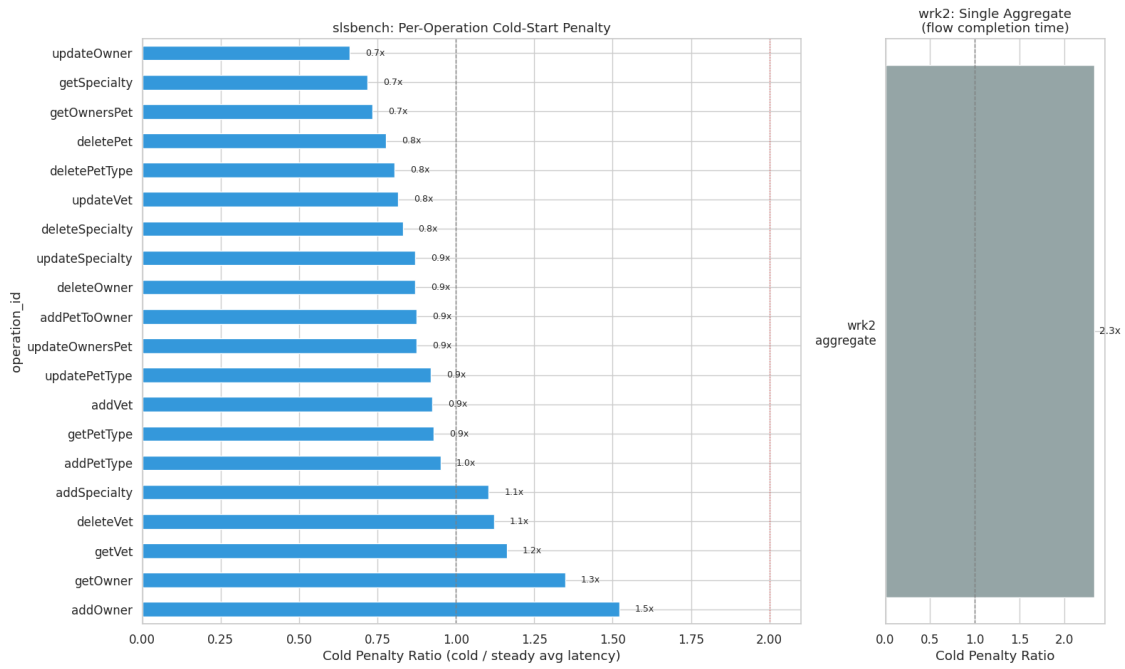
1.3 TP-B: Execution Phase Granularity

Hypothesis: Cold-start penalties differ per-operation, and scenario-based benchmarks expose this while aggregate wrk2 measurements report a single blended value.

Independent variable: Execution phase (cold vs steady)

Control: scenario=mixed, rate=500, app=spring

Chart B1: Cold-Start Penalty -- Per-Op (sisbench) vs Aggregate (wrk2)

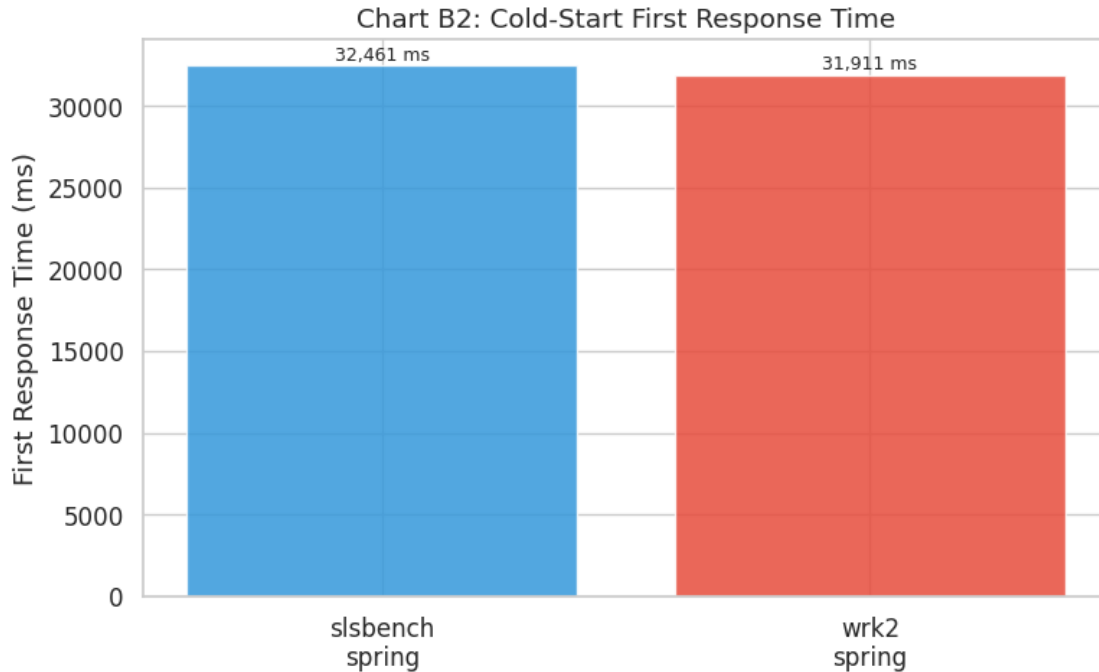


Per-operation cold penalty range: 0.7x to 1.5x

Operations with penalty >2x: 0 of 20

First-Response Time Comparison (cold phase)

		mean_ms	std_ms	count
treatment	app			
sisbench	spring	32461.0	NaN	1
wrk2	spring	31911.0	NaN	1



wrk2 Aggregate: cold vs steady (mixed scenario)

Note: wrk2 latencies represent flow completion times, not individual request latency.

	avg_latency_ms	p99_latency_ms	throughput	non_2xx
phase				
cold	10610.0	12300.0	89.44	224
steady	4540.0	6610.0	394.56	1973

COLD PENALTY VARIABILITY: max=1.5x, min=0.7x, spread=2.3x (threshold: >1.5x)

-> CONFIRMED: Cold-start penalties differ per operation

WRK2 VISIBILITY:

wrk2 cold avg: 10610.0 ms (flow completion time)

wrk2 steady avg: 4540.0 ms (flow completion time)

wrk2 ratio: 2.3x (single aggregate)

slsbench reveals 20 distinct penalties: 0.7x -- 1.5x

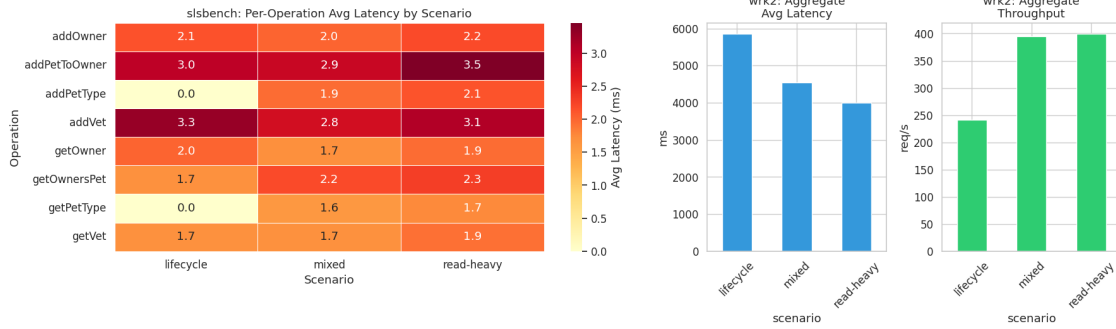
1.4 TP-C: Scenario Shape Sensitivity

Hypothesis: Different scenario shapes (read-heavy vs mixed vs lifecycle) produce measurably different per-operation latency profiles, while wrk2 reports similar aggregates across all scenarios.

Independent variable: Scenario (read-heavy, mixed, lifecycle)

Control: app=spring, phase=steady, rate=500

TP-C: Scenario Shape Sensitivity -- slsbench per-op detail vs wrk2 aggregate



Coefficient of Variation per operation (ops present in all 3 scenarios):

getOwnersPet: CV = 0.158
 addPetToOwner: CV = 0.096
 getOwner: CV = 0.088
 addVet: CV = 0.084
 getVet: CV = 0.079
 addOwner: CV = 0.048

SCENARIO SENSITIVITY CHECK:

Mean CV across scenarios: 0.092 (threshold: >0.15)
 -> NOT CONFIRMED: Scenario shapes produce different profiles

wrk2 aggregate CV: 0.200
 slsbench per-op mean CV: 0.092
 -> NOT CONFIRMED: slsbench reveals more sensitivity than wrk2

1.5 TP-D: Cross-Framework Decision Quality

Hypothesis: Per-operation metrics from scenario-based benchmarks enable more informed framework selection than aggregate wrk2 latency numbers.

Independent variable: Framework (Spring, Quarkus JVM)

Control: scenario=lifecycle, phase=steady, rate=500

Table D1: slsbench Per-Operation Avg Latency by Framework (lifecycle, steady)

	quarkus-jvm (us)	spring (us)
operation_id		
addOwner	1087.0	2141.5
addPetToOwner	NaN	3011.0
addVet	1023.0	3299.0
addVisitToOwner	NaN	2228.5
getOwner	NaN	2014.0
getOwnersPet	NaN	1682.5
getVet	NaN	1677.0

getVisit	NaN	1980.5
listPets	NaN	13806.5
listVets	1019.5	4244.0
listVisits	NaN	10091.0

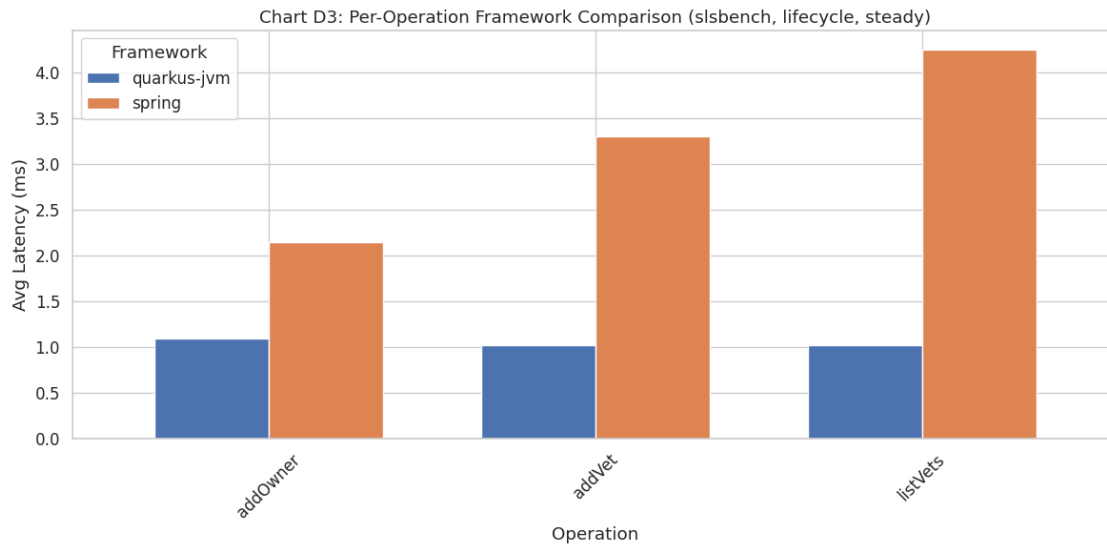
quarkus-jvm faster on 3/3 operations

spring faster on 0/3 operations

Table D2: wrk2 Aggregate Comparison by Framework

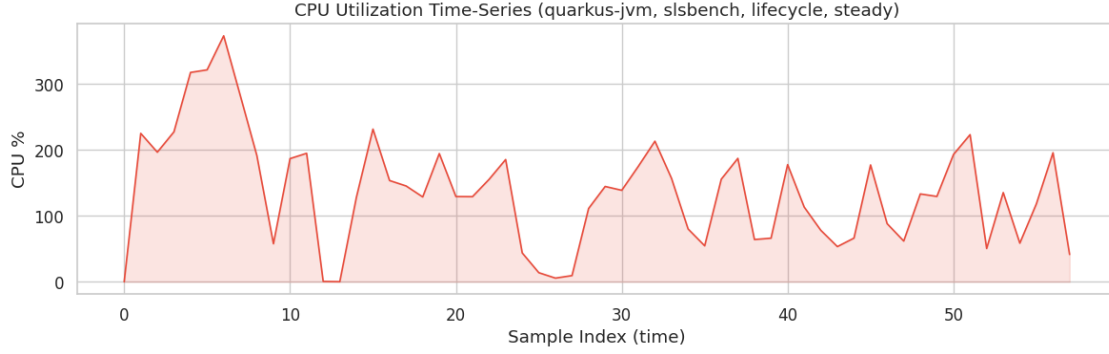
Note: wrk2 latency = flow completion time (all chain steps combined)

	avg_latency_ms	p99_ms	throughput_rps	non_2xx
app				
quarkus-jvm	11655.0	18245.0	111.18	0
spring	5860.0	8320.0	241.56	2415



Container Resource Utilization Summary:

treatment	app	scenario	phase	avg_cpu	max_cpu	avg_mem	n_runs
slsbench	quarkus-jvm	lifecycle	steady	188.8	481.3	1.7	1
	spring	lifecycle	steady	393.1	945.8	2.3	1
		mixed	cold	409.2	785.6	1.9	1
			steady	429.3	790.8	2.2	1
		read-heavy	steady	417.4	907.8	2.3	1



DECISION QUALITY CHECK:

```
addOwner: best = quarkus-jvm (97.0% margin)
addVet: best = quarkus-jvm (222.5% margin)
listVets: best = quarkus-jvm (316.3% margin)
```

```
quarkus-jvm wins 3/3 operations
```

```
spring wins 0/3 operations
```

-> quarkus-jvm dominates all operations, but per-op data shows margin variability.

```
wrk2 aggregate winner: spring (flow completion time)
```

```
wrk2 provides 1 ranking. slsbench reveals 3 operation-level rankings.
```

1.6 5. Summary and Thesis Conclusions

This section maps the experimental results to the thesis abstract claims and provides a structured hypothesis confirmation summary.

Abstract Claim	Testing Point	Key Evidence
Realistic usage patterns > isolated micro-measurements	TP-A	Per-operation variance, saturation curves, error rates
Model execution phases (cold start, steady state)	TP-B	Per-operation cold penalty ratios vs wrk2 single aggregate
Different scenario shapes yield different insights	TP-C	Latency heatmap CV across scenarios vs wrk2 flat aggregates
Comparative insights across frameworks/runtimes	TP-D	Per-operation framework ranking vs wrk2 single winner
Improved decision-making vs microbenchmarks	All TPs	Side-by-side wrk2 aggregate vs slsbench per-op comparison

=== Hypothesis Confirmation Summary ===

<pandas.io.formats.style.Styler at 0x74d955aba1a0>

Results: 5 confirmed, 1 not confirmed, 0 pending

1.7 6. Full Benchmark Commands

The cells above work with whatever data exists in `results/`. To populate the full evaluation dataset, execute the commands below from the `evaluation/` directory.

Prerequisites: 1. Docker images: `eval-wrk2:latest`, `aaape2k/spring-petclinic-rest`, `aaape2k/quarkus-petclinic-jvm`, `slsbench:dood`, `aaape2k/wrk2-flow:v3.0` 2. Probe-bodies: `./scripts/run-probe-all.sh --apps spring,quarkus --scenarios read-heavy,mixed,lifecycle`

```
# =====
# Full Benchmark Execution Commands
# Run from: harness-evaluation/evaluation/
# =====

# --- Prerequisites ---
# Generate probe-bodies (once per flow):
RATES=50 ./scripts/run-probe-all.sh --apps spring --scenarios
lifecycle,mixed,read-heavy
RATES=50 ./scripts/run-probe-all.sh --apps quarkus-jvm --scenarios lifecycle

# --- TP-A: Per-Operation Performance Characterization ---
./scripts/run-all.sh --apps spring,quarkus-jvm --scenarios lifecycle --phases
steady --reps 3

# --- TP-B: Execution Phase Granularity ---
./scripts/run-all.sh --apps spring,quarkus-jvm --scenarios mixed --phases
cold,steady --reps 3 --rates 500

# --- TP-C: Scenario Shape Sensitivity ---
./scripts/run-all.sh --apps spring --scenarios read-heavy,mixed,lifecycle
--phases steady --reps 2 --rates 500

# --- TP-D: Cross-Framework Decision Quality ---
./scripts/run-all.sh --apps spring,quarkus-jvm --scenarios lifecycle --phases
cold,steady --reps 3 --rates "200 500"

# --- Quick Sample (for validation) ---
./scripts/run-all.sh --apps spring --scenarios lifecycle --phases steady --reps
1 --rates "200 500"
./scripts/run-all.sh --apps spring --scenarios mixed --phases cold,steady --reps
```

```
1 --rates 500
./scripts/run-all.sh --apps spring --scenarios read-heavy --phases steady --reps
1 --rates 500
./scripts/run-all.sh --apps quarkus-jvm --scenarios lifecycle --phases steady
--reps 1 --rates 500
```